JYX JYVÄSKYLÄN YLIOPISTO UNIVERSITY OF JYVÄSKYLÄ

This is a self-archived version of an original article. This version may differ from the original in pagination and typographic details.

Author(s): Sami, Malik Abdul; Waseem, Muhammad; Zhang, Zheying; Rasheed, Zeeshan; Systä, Kari; Abrahamsson, Pekka

Title: Early Results of an AI Multiagent System for Requirements Elicitation and Analysis

Year: 2025

Version: Accepted version (Final draft)

Copyright: © The Editor(s) (if applicable) and The Author(s), under exclusive license to Springe

Rights: In Copyright

Rights url: http://rightsstatements.org/page/InC/1.0/?language=en

Please cite the original version:

Sami, M. A., Waseem, M., Zhang, Z., Rasheed, Z., Systä, K., & Abrahamsson, P. (2025). Early Results of an AI Multiagent System for Requirements Elicitation and Analysis. In D. Pfahl, J. G. Huerta, J. Klünder, & H. Anwar (Eds.), Product-Focused Software Process Improvement : 25th International Conference, PROFES 2024, Tartu, Estonia, December 2–4, 2024, Proceedings (pp. 307-316). Springer. Lecture Notes in Computer Science. https://doi.org/10.1007/978-3-031-78386-9_20

Early Results of an AI Multiagent System for Requirements Elicitation and Analysis

Malik Abdul Sami[1[0000−0001−5136−2587]], Muhammad Waseem[2[0000−0001−7488−2577]], Zheying Zhang[1[0000−0002−6205−4210]], Zeeshan Rasheed[1[0000−0001−9655−3096]], Kari Systä[1[0000−0001−7371−0773]], and Pekka Abrahamsson[1[0000−0002−4360−2226]]

[1] Tampere University, Tampere, Finland
malik.sami@tuni.fi, zheying.zhang@tuni.fi, zeeshan.rasheed@tuni.fi, kari.systa@tuni.fi, pekka.abrahamsson@tuni.fi
[2] University of Jyväskylä, Jyväskylä, Finland
muhammad.m.waseem@jyu.fi

Abstract. In agile software development, user stories capture requirements from the user's perspective, emphasizing their needs and each feature's value. Writing concise and quality user stories is necessary for guiding software development. Alongside user story generation, prioritizing these requirements ensures that the most important features are developed first, maximizing project value. This study explores the use of Large Language Models (LLMs) to automate the process of user story generation, quality assessment, and prioritization. We implemented a multi-agent system using Generative Pre-trained Transformers (GPT), specifically GPT-3.5 and GPT-4o, to generate and prioritize user stories from the initial project description. Our experiments on a real-world project demonstrate that GPT-3.5 handled user story generation well, achieving a higher semantic similarity score comnpared to the GPT-4o. Both models show consistent performance in prioritizing requirements and identify the core features of the application. These early results indicate that LLMs can automate requirements analysis, especially in generating and prioritizing user stories

Keywords: Software Engineering, Requirements Engineering, Requirements prioritization, Generative AI, Large language model(s)

1 Introduction

Requirements are often specified as user stories in agile software projects [2], which describe functionalities from the user's perspective, highlighting their needs and the value provided. Specifying and prioritizing user stories is an ongoing process, conducted through close communication between users and developers in non-technical language [4]. Effective user stories are crucial, as they guide the development process by conveying user needs. Poorly specified stories can introduce bias, be prone to errors, and often lack clarity [3].

Recent advances in Natural Language Processing (NLP) have led to transformative applications across various domains, such as healthcare, cybersecurity, e-commerce, etc. [13]. Among these, LLMs have emerged as powerful tools in requirements analysis tasks, offering the potential to automate requirements development through efficient elicitation, specification, and prioritization [6]. LLMs have also shown progress in addressing complex software development challenges by simulating various development roles [7] [9] [11]. Studies have explored multi-agent integration within LLM frameworks for tasks like software design and code generation in the Software Development Life Cycle (SDLC) and prioritization techniques to prioritize stories to design better software [10]. Although Zhang et al. [14] reported initial findings on the use of LLMs as agents for enhancing user story qualities, studies on integrating multi-agent LLMs into the RE process, particularly generating and managing user stories, remain limited.

While LLMs have been explored for specific requirements tasks, less focus is on using a multi-agent approach to prioritizing user stories. Our work addresses this gap by implementing and evaluating a multi-agent system for generating and prioritizing user stories with multiple AI agents involvement. We designed two research questions (RQs), as follows:

- **RQ1**. How can the proposed multi-agent system assist in requirements analysis, specifically in user story generation and prioritization?
- **RQ2**. How effectively can the multi-agent system prioritize these user stories?

To address RQs, we developed a multi-agent system to assist in generating user stories and enhance their description following the INVEST [1], a quality framework widely used in agile projects as a guideline for ensuring the quality of user stories. It includes attributes such as independent, negotiable, valuable, estimable, small, and testable. In the requirements prioritization task, we applied the 100 dollar prioritization technique [12] to rank the user stories and analyze the generated results.

The contributions of this study are twofold: a) the development of the first AI-driven multi-agent system for user story generation and specification using a given INVEST quality framework; and b) the implementation of user story prioritization, addressing gaps in stakeholder involvement during the prioritization process.

The rest of this paper is structured as follows: Section 2 outlines the study methodology, and Section 3 details the implementation and experiment design. Section 4 contains the early results, and Sections 5 and 6 provide discussion and conclusions.

2 Proposing Agents for Requirements Analysis

To address the RQs, we designed a multi-agent system using LLMs as agents to automate a requirements analysis process. As shown in Fig. 1, the LLM agents

simulate the roles and tasks typically performed in agile teams for software development, automating activities of user story generation, quality assurance, and prioritization. A breakdown of how the agents function in our implementation is depicted in Table 1.

The Product Owner (PO) agent plays the role of a PO. In agile practice, the PO is responsible for understanding stakeholders' requirements and converting them into user stories as items in the project backlog. Here, the PO agent automates that task by creating user stories based on the provided project description to describe functionalities from the user's perspective.

After the user stories are generated, the Quality Assurance (QA) agent checks and enhances the quality of these stories. This reflects the work of QA specialists who review user stories to ensure they adhere to the organization's quality standards, such as the INVEST framework [1]. The QA agent thus helps maintain the quality and effectiveness of the user stories before they move forward in the development process.

The Developer (Dev) agent and manager agent collectively prioritize the user stories with PO and QA agents. In a typical agile setting, developers and managers collaborate to prioritize tasks based on project goals, technical constraints, and stakeholder input. The agents here replicate this decision-making process, using input from both PO and QA agents to rank user stories effectively. This automated prioritization is similar to stakeholder-driven planning meetings, ensuring that development teams focus on the most important or valuable features first.

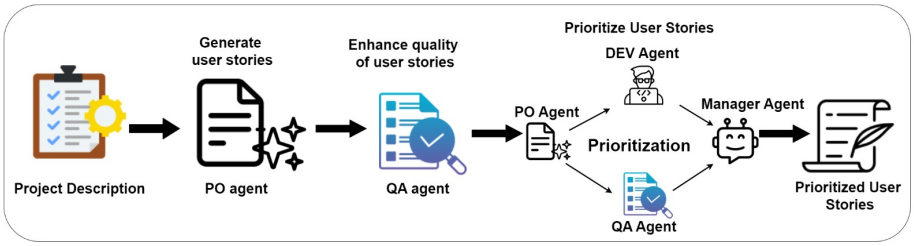

Fig. 1. Process Model of LLM Agents for User Story Generation, Quality Check, and Prioritization.

3 Implementation and Experimental Design

We implemented a multi-agent system to evaluate the effectiveness of using GPT-3.5 and GPT-4 Omni as agents in automating the requirements analysis activities. Specifically, we focused on generating user stories, assessing their quality using the INVEST framework [5], and prioritizing them using the $100 prioritization technique. For prioritization, the $100 prioritization method is

Table 1. Tasks and prompts summary of the four Agents and their tasks and roles

#	Agent	Tasks	Described Roles
1	PO Agent	User Story Generation and Communication, Prioritization of Stories	This prompt guides an assistant in creating detailed user stories and corresponding StoryNames from project descriptions, focusing on unique, functional, and technical aspects and prioritizing these user stories in their backlog using specified techniques.
2	QA Agent	Quality Assurance Assessment and Prioritization of Stories	This prompt instructs an assistant to evaluate user stories against a specified framework, assessing quality and identifying issues. It also ensures the prioritization of requirements by understanding the context and ensuring coverage and effectiveness.
3	Dev Agent	Prioritizing User Stories	Acts as a senior developer to understand project requirements and suggest the prioritization of the requirements by understanding the context.
4	Manager Agent	Combining and Ranking Final Prioritization	This prompt instructs an assistant to prioritize user stories $100, on the provided context and previous discussions. The assistant is tasked with allocating points or prioritizing features to reflect their relative importance, ensuring the ranking adheres to the required format and is supported by detailed justifications.

used. In this implementation, the PO agent, Dev agent, and QA agent each allocate 100 points to user stories based on their perceived importance. The final prioritization is synthesized by the Manager agent. This process ensures that the most critical requirements are ranked the highest, based on both technical and quality considerations from those agents, so that the development team can focus on the most impactful features in the development project.

Our experiment is built around a real-world project, Supplier Data Management Platform (SDMP), to simulate the practical application of the AI models in handling requirements analysis tasks. The project aims to develop a new platform to streamline data extraction from PDFs, addressing the challenge of managing and comparing data from numerous vendors. The current system struggles with handling vendor diversity and tracking cost effectively. The new platform automates data extraction, translation, and formatting, which is then stored in MongoDB. This enables real-time analysis to provide insights into the data. The main features include identifying and comparing pricing information from various suppliers, recommending the best suppliers based on the data, and calculating the total cost of procurement.

3.1 System Design

The web-based application is developed using *HTML*, *CSS*, *JavaScript*, *React*, and *WebSocket* on the frontend, with *Python* and the *Starlette* framework on the backend. *WebSocket* is used for real-time communication, while HTTP requests facilitate interaction with *GPT-3.5* and *GPT-4o* models. As illustrated in Figure 2, WebSocket handles real-time interactions between the frontend and backend, which is essential for tasks requiring immediate feedback, such as prioritization using the $100 prioritization technique. For tasks like generating user stories, the

frontend communicates with the backend via API calls to OpenAI's GPT-3.5 and GPT-4o models. The responses, typically in natural language, are filtered and processed to extract relevant information. Once the user stories are generated, the system evaluates their quality using the *INVEST* framework, ensuring they are Independent, Negotiable, Valuable, Estimable, Small, and Testable. For prioritization tasks, the system uses the $100 prioritization technique to allocate resources to tasks or user stories based on perceived value. The source code of the tool and the experiment-related materials are available on GitHub[3].

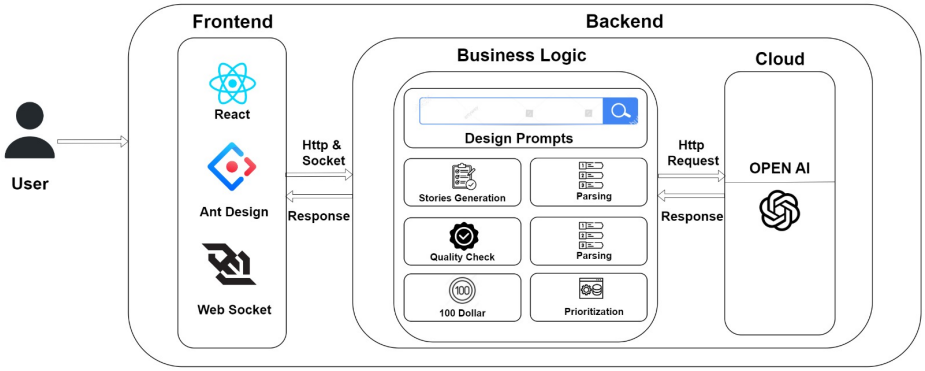

Fig. 2. Architecture of the Multi-Agent System

3.2 Evaluation

We evaluate the generated user stories by analyzing their semantic similarity to the project descriptions, i.e. the input to the multi-agent system. Specifically, we employ the sentence transformer model all-MiniLM-L6-v2 [8], which converts sentences into embedding that capture their semantic meaning. The embeddings are then used for tasks such as semantic similarity, clustering, and classification. We calculate the semantic similarity score for each user story by measuring how closely it matches the project's description. In addition to semantic matching, we also assess whether each user story adheres to the specified format for quality checks. This evaluation ensures that the stories meet the required quality standards.

Regarding the evaluation of generated user story priority, we assess the consistency of the priority rankings generated by the two models, by identify the optimal prioritization which is the ranking most frequently generated by the agents, using $100 prioritization technique. These calculations are performed on a Google Colab CPU instance[4]. To visualize and compare these metrics effectively, a Python script is crafted using pandas and matplotlib, with the resulting

[3] https://github.com/GPT-Laboratory/multiagent-prioritization
[4] https://colab.research.google.com/

plots providing a side-by-side comparison of the model's performance across the projects. This enables us to display and compare the outcomes generated by different LLM models acting as agents, using $100 prioritization technique.

4 Preliminary Results

We conducted experiments to generate user stories and prioritize them based on the provided description of the SDMP project, using GPT-3.5 and GPT-4o as agents separately. The results are presented in Fig. 3, with metrics such as the number of stories generated, API response time, and similarity scores with the project descriptions.

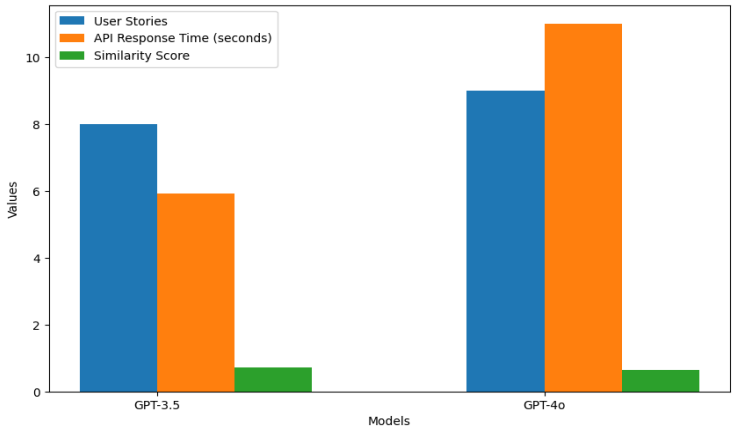

Fig. 3. Comparison of GPT-3.5 and GPT-4o: User Stories, response time, and semantic similarity scores

As shown in Fig. 3, GPT-3.5 generates eight user stories, resulting in a high similarity score of 0.72 and an API response time of 5.91 seconds. In contrast, GPT-4o achieves a substantial API response time of 11.00 seconds and a similarity score of 0.65, while managing nine user stories. The user stories generated by GPT-3.5 were subsequently prioritized using both GPT-3.5 and GPT-4o agents. The results are presented in Table 2. Additional details, including raw data, the analysis process, agent interactions, and the story prioritization technique, are accessible on GitHub[5].

Both GPT-3.5 and GPT-4o models largely agree on the user story priorities, such as the PDF Extraction and Translation feature (Story 1) and Data Analysis and Insights (Story 2), which are crucial for the system's core functionality. However, there are slight differences in the perceived importance of other features

[5] https://github.com/GPT-Laboratory/multiagent-prioritization

Table 2. Comparison of user story prioritization by GPT-3.5 and GPT-4o (Based on GPT-3.5 Stories)

#	Story Name	User Story	GPT-3.5 Rank	GPT-3.5 $100	GPT-4o Rank	GPT-4o $100
1	PDF Extraction and Translation	As a procurement manager, I want the chatbot to extract data from PDFs of different vendors and translate them into English, to streamline the process of comparing prices across 500 suppliers.	1	20	1	25
2	Data Analysis and Insights	As a supply chain analyst, I want the chatbot to analyze menu costs and resource costs from different vendors, providing insights to help in decision-making for profitable deals.	2	15	2	20
3	Supplier Rate Comparison	As a procurement officer, I want the chatbot to compare supplier rates for specific ingredients needed to make lasagna, to determine the most cost-effective option.	4	12	5	10
4	Recipe Analysis and Cost Calculation	As a chef, I want the chatbot to provide the recipe and cost analysis for making 20 plates of lasagna, including ingredient quantities and total cost, for efficient meal planning.	6	10	8	2
5	Database Integration	As a technical lead, I want the chatbot to integrate with a vector database like MongoDB to store structured data, enabling efficient access and utilization by the Large Language Model (LLM).	3	13	3	20
6	Demo Development	As a project manager, I want the chatbot to deliver a functional demo by the third week of August, showcasing its capabilities in data extraction, analysis, and providing insights to stakeholders.	5	10	4	15
7	API Development	As a software developer, I want to provide APIs for the team to upload PDFs and perform detailed insights on historical data and various offers, enhancing data analysis capabilities.	5	10	6	5
8	Advanced Data Analysis	As a data analyst, I want the chatbot to enable deeper analysis of historical data and trends, leveraging the Large Language Model (LLM) integration to optimize procurement strategies and cost management.	5	10	7	3

like API Development (Story 7) and Recipe Analysis and Cost Calculation (Story 4). GPT-3.5 ranks API Development higher and allocates $10, while GPT-4o ranks it lower with only $5, implying a divergence in how each model values system extensibility. Similarly, Recipe Analysis is ranked significantly lower by GPT-4o (8th with $2) than GPT-3.5 (6th with $10). Consensus on priorities like PDF Extraction suggests focusing early development efforts there, while the divergence on other features can inform future strategic decisions based on the project's goals, such as immediate core functionality versus broader system capabilities.

5 Discussion

Our initial finding demonstrates the potential of multi-agent systems and LLMs in automating requirements engineering tasks. Our approach simplifies the requirements analysis process through the collaboration of agents, using GPT-3.5 and GPT-4o. The results indicate that this method can increase productivity and precision in requirements engineering tasks, though further experimentation is required to refine the prompts and integrate human experts such as product owners and quality assurance specialists into the interaction loop with autonomous agents.

AI-driven Multi-Agent Assistance in Requirements Management: Our experiments show that multi-agent systems streamline and enhance requirements analysis. The PO agent generates user stories and initiates discussions, while the QA agent assesses their quality. The Senior Developer and QA agents contribute to

prioritizing user stories based on their understanding of project requirements, and the Manager agent finalizes the prioritization by aggregating inputs from other agents. This collaborative system enabled efficient parsing and analysis of requirements, leading to a more systematic and comprehensive needs analysis. *Story Name and User Stories:* GPT-3.5 consistently generated more Story Names and maintained higher similarity scores, proving its robustness in handling complex projects. However, GPT-4o produced more user stories but with longer API response times. *API Response Time:* GPT-3.5 achieved shorter response times, enhancing efficiency for real-time requirements generation tasks. *Effectiveness of LLMs in Ranking User Stories:* Using the $100 prioritization technique, GPT-3.5 and GPT-4o produced reliable ranking results. GPT-4o allocated more resources to critical tasks like data extraction, while GPT-3.5 distributed resources more evenly, highlighting its balanced approach to prioritization. *Prioritization Technique:* The $100 method consistently delivered strong prioritization results, emphasizing key objectives and practical applications. Both models allocated the highest resources to foundational tasks like PDF extraction and translation.

Our system promises to generate and rank stories. However, challenges include agents speaking out of context, lengthy inputs and prompts causing API failures, and occasional parsing errors due to unstructured responses, and this requires proper resource profiling. The issue of generating new requirements every time needs to be addressed and made persistent. Hallucinations could be mitigated through the integration of Retrieval-Augmented Generation (RAG) techniques. Prompts need to be refined, as some technical aspects are still missed when generating stories. Feedback from practitioners is necessary to assess the system's acceptance. Future work should explore additional LLM models, fine-tune them for user story generation and prioritization, and introduce other prioritization techniques for deeper insights.

6 Conclusion

In this study, we developed a multi-agent system that generates user stories, assesses their quality, and prioritizes them. We conducted multiple experiments to evaluate their results and ranking with $100 technique and two LLMs mdoels. The use of four distinct AI agents—PO, QA, Dev, and Manager Agent enabled us to generate user stories, facilitate stakeholder discussions, and evaluate quality and compliance. GPT-3.5 consistently achieved higher similarity scores, while GPT-4o exhibited higher API response times but excelled in user story generation and prioritization.

We further assessed the role of the LLM models on requirements prioritization tasks. GPT-3.5 showed a balanced approach, distributing resources more evenly across all user stories, ensuring main functionality. In contrast, GPT-4o ranked foundational features like data extraction and integration, allocating more resources to high-impact tasks.

References

1. Wake, b: Invest in good stories, and smart tasks. https://xp123.com/articles/invest-in-good-stories-and-smart-tasks/ (Accessed: 2024-1-10)
2. Abed, O., Nebe, K., Abdellatif, A.B.: Ai-generated user stories supporting human-centred development: An investigation on quality. In: Stephanidis, C., Antona, M., Ntoa, S., Salvendy, G. (eds.) HCI International 2024 Posters. pp. 3–13. Springer Nature Switzerland, Cham (2024)
3. Alawaji, B., Hakami, M., Alshemaimri, B.: Evaluating generative language models with prompt engineering for categorizing user stories to its sector domains. In: 2024 IEEE 9th International Conference for Convergence in Technology (I2CT). pp. 1–8. IEEE (2024)
4. Herwanto, G.B.: Automating data flow diagram generation from user stories using large language models. In: 7th Workshop on Natural Language Processing for Requirements Engineering (2024)
5. ISO/IEC/IEEE International Standard 29148:2011(E): Iso/iec/ieee international standard - systems and software engineering – life cycle processes – requirements engineering. ISO/IEC/IEEE 29148:2011(E) pp. 1–94 (2011). https://doi.org/10.1109/IEEESTD.2011.6146379
6. Mehraj, A., Zhang, Z., Systä, K.: A tertiary study on ai for requirements engineering. In: International Working Conference on Requirements Engineering: Foundation for Software Quality. pp. 159–177. Springer (2024)
7. Rasheed, Z., Waseem, M., Saari, M., Systä, K., Abrahamsson, P.: Codepori: Large scale model for autonomous software development by using multi-agents. arXiv preprint arXiv:2402.01411 (2024)
8. Reimers, N., Gurevych, I.: Sentence-bert: Sentence embeddings using siamese bert-networks. arXiv preprint arXiv:1908.10084 (2019)
9. Sami, A.M., Rasheed, Z., Kemell, K.K., Waseem, M., Kilamo, T., Saari, M., Duc, A.N., Systä, K., Abrahamsson, P.: System for systematic literature review using multiple ai agents: Concept and an empirical evaluation. arXiv preprint arXiv:2403.08399 (2024)
10. Sami, M.A., Waseem, M., Rasheed, Z., Saari, M., Systä, K., Abrahamsson, P.: Experimenting with multi-agent software development: Towards a unified platform. arXiv preprint arXiv:2406.05381 (2024)
11. Waseem, M., Das, T., Paloniemi, T., Koivisto, M., Räsänen, E., Setälä, M., Mikkonen, T.: Artificial intelligence procurement assistant: Enhancing bid evaluation. In: International Conference on Software Business. pp. 108–114. Springer Nature Switzerland Cham (2023)
12. Wiegers, K.E., Beatty, J.: Software requirements. Pearson Education (2013)
13. Xie, Z., Chen, Y., Zhi, C., Deng, S., Yin, J.: Chatunitest: a chatgpt-based automated unit test generation tool. arXiv preprint arXiv:2305.04764 (2023)
14. Zhang, Z., Rayhan, M., Herda, T., Goisauf, M., Abrahamsson, P.: Llm-based agents for automating the enhancement of user story quality: An early report. In: International Conference on Agile Software Development. pp. 117–126. Springer Nature Switzerland Cham (2024)